

RNCP36739 — Expert en Ingénierie de Données

Bloc 3

Implémenter et optimiser des solutions de stockage et de traitement de données sur le cloud

Projet support : **project_cloud** — plateforme de mobilité lyonnaise en temps réel

Julien TREMONT-RAIMI • Antoine VANDEPLANQUE • Promotion 2024–2026

Cette partie démontre l'acquisition des quatre compétences du Bloc 3 du référentiel RNCP36739 [1] à travers une plateforme **cloud serverless AWS**. Chaque étape couvre une ou plusieurs sous-compétences du référentiel, synthétisées dans le tableau de validation à la fin du bloc.

1 Compétences visées

Le référentiel RNCP36739 définit quatre sous-compétences pour le Bloc 3, que ce dossier couvre intégralement à travers le **projet cloud**. Ce bloc porte sur la mise en œuvre de solutions cloud pour le stockage, le traitement et la mise à disposition des données : choix des services managés, conception des flux, sécurisation des accès, optimisation de la performance et de la disponibilité

- Mettre en œuvre des solutions de stockage de données dans le cloud pour permettre aux entreprises d'explorer leurs données en implémentant des techniques et stratégies adaptées à leur utilisation.
- Concevoir et développer des pipelines et des solutions de traitement de données dans le cloud en chargeant les données, en les transformant et en les mettant à disposition des utilisateurs.
- Mettre en place une politique de sécurité des données dans le cloud en développant une stratégie de chiffrement et de gestion des identités et des accès, dans le respect des règlements en vigueur.
- Optimiser les solutions de stockage et de traitement des données dans le cloud en définissant des indicateurs de performance pour assurer la disponibilité des services et optimiser les coûts.

2 Contexte et enjeux

Le **projet cloud** est une plateforme de données temps réel lyonnaise. Elle agrège en continu trois flux de données ouvertes : les départs ferroviaires et de métro en gare de Lyon (SNCF/RATP), l'état des parkings parisiens (disponibilité, places, tarifs) et l'état du trafic routier (indice de congestion, segments critiques, fluidité par zone). L'objectif est de permettre un suivi opérationnel de la mobilité lyonnaise via un tableau de bord Streamlit alimenté par une API serverless.

L'enjeu architectural central est double : garantir la latence sur des données en temps réel et maîtriser les coûts. Un scan DynamoDB complet pour chaque requête API serait fonctionnellement correct, mais son coût de lecture croîtrait linéairement avec le volume. La mise en place d'un Global Secondary Index (GSI) ramène le coût de chaque requête à $O(1)$, quelle que soit la taille de la table.

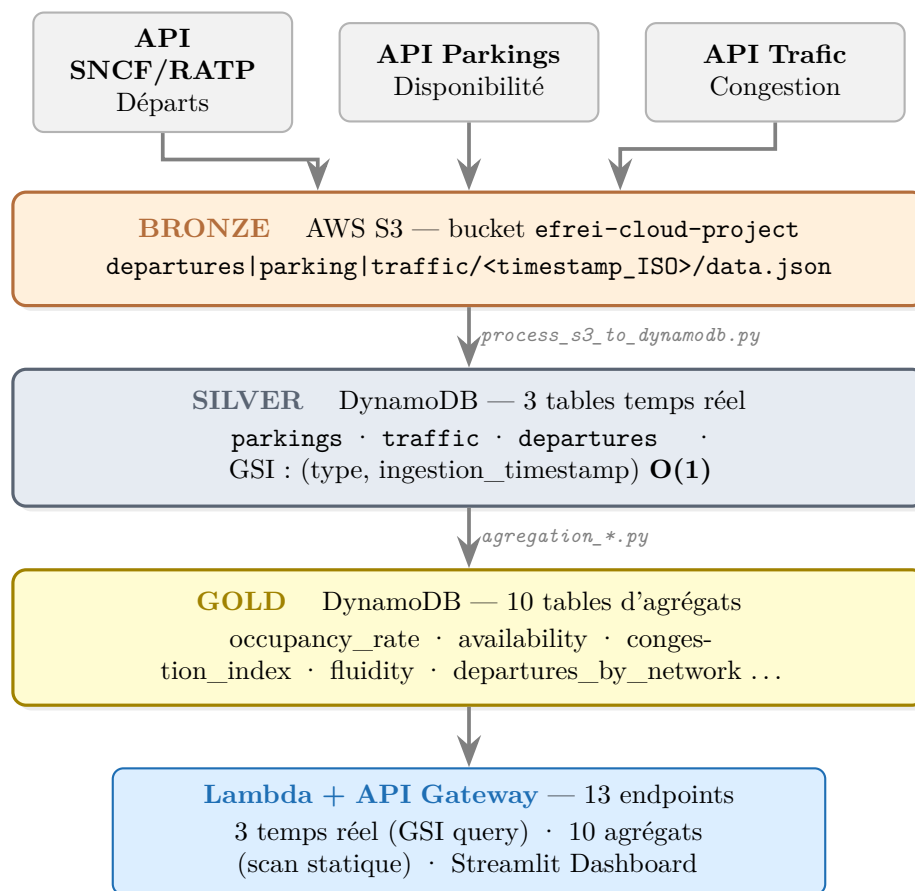
2.1 Positionnement dans l'écosystème AWS

AWS propose plusieurs alternatives pour chaque composant de l'architecture. Les choix retenus sont justifiés par rapport à ces alternatives.

Besoin	Choix retenu	Alternative	Justification
Lac de données	S3	EFS, EBS	Coût minimal, accès objet universel
Base NoSQL	DynamoDB	MongoDB Atlas, ElasticSearch	Managé, serverless, GSI natif
Compute API	Lambda	EC2, ECS/Fargate	Pas d'infra à gérer, facturation à l'usage
Routage API	API Gateway	ALB, CloudFront	Intégration Lambda native, gestion CORS
Dashboard	Streamlit (EC2)	QuickSight, PowerBI	Open source, Python natif
Scheduling	EventBridge	Step Functions, MWAA	Cron simple, coût négligeable

3 Architecture cloud

L'architecture suit le motif Medallion (Bronze/Silver/Gold) transposé sur des services managés AWS : S3 comme data lake brutes, DynamoDB comme couche nettoyée puis agrégée, et une exposition serverless via Lambda et API Gateway.



Le schéma technique du projet ci-dessous détaille le flux complet, du calcul ordonnancé sur EC2 jusqu'à la visualisation Streamlit.

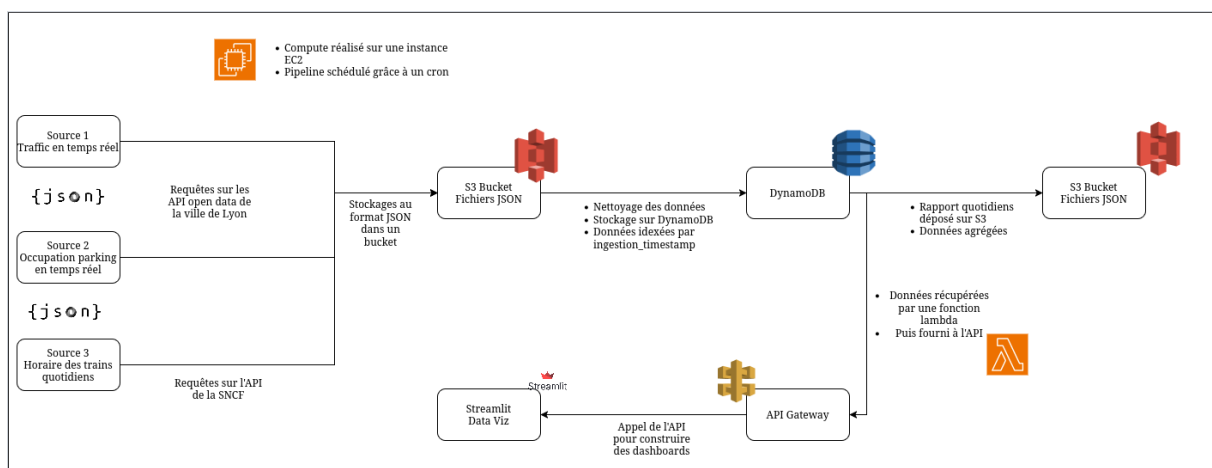


Figure 1 – Architecture technique du *projet cloud*. Les trois sources open data sont collectées par un pipeline ordonnancé sur EC2, stockées en JSON dans S3, nettoyées et indexées dans DynamoDB, agrégées dans un rapport quotidien, puis exposées via une fonction Lambda et une API Gateway consommée par le dashboard Streamlit.

4 Stockage S3 — data lake brutes horodaté

AWS S3 [2] constitue la couche Bronze de l'architecture. Chaque ingestion produit un objet JSON horodaté dans le préfixe correspondant à la source (`departures/<timestamp_ISO>/data.json`). Cette organisation préserve l'historique complet des snapshots et permet de rejouer n'importe

quelle fenêtre temporelle sans perte.

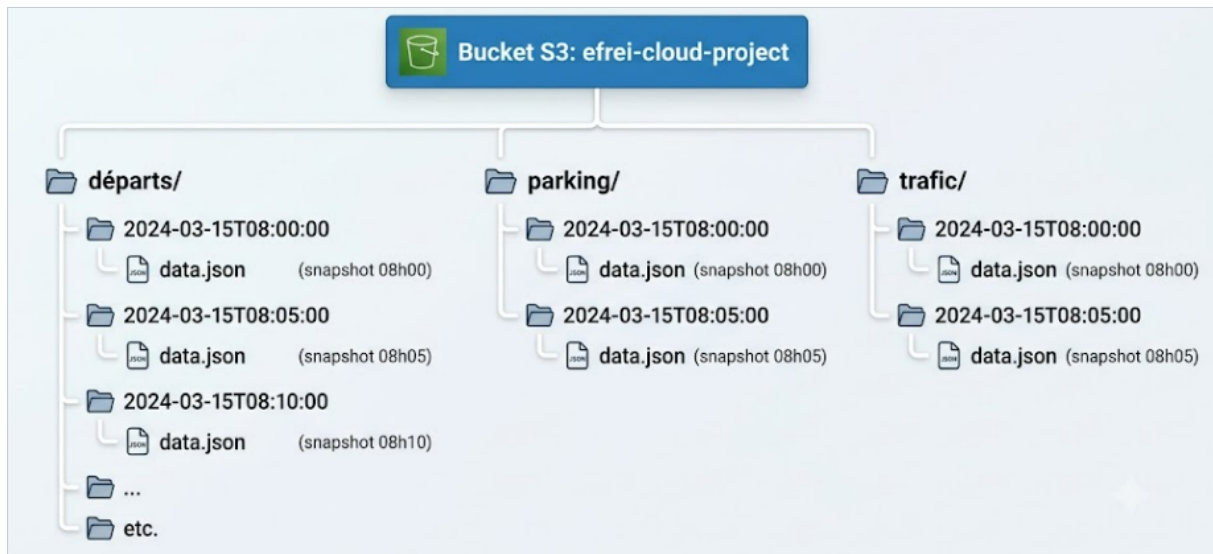


Figure 2 – Organisation du bucket S3 *efrei-cloud-project* : un préfixe par source (départs, parking, trafic) et, sous chacun, un dossier horodaté par snapshot contenant le fichier *data.json*.

```
def upload_to_s3(s3, json_data, bucket_name, key):
    s3.put_object(
        Bucket=bucket_name,
        Key=key,
        Body=json.dumps(json_data, indent=4, cls=DecimalEncoder)
    )

def get_json_from_s3(s3, bucket, key):
    response = s3.get_object(Bucket=bucket, Key=key)
    content = response["Body"].read().decode("utf-8")
    return json.loads(content)

def get_s3_object_keys(s3, bucket_name, prefix):
    response = s3.list_objects_v2(Bucket=bucket_name, Prefix=prefix)
    return [obj['Key'] for obj in response.get('Contents', [])]
```

Extrait 1 – Écriture et lecture S3 (utils_s3.py)

Le `DecimalEncoder` sérialise les `Decimal` Python en `int` ou `float` selon la présence d'une partie décimale — une nécessité, car DynamoDB stocke tous les nombres au format `Decimal`.

[Capture] Console AWS S3 – bucket efrei-cloud-project, dossier departures : liste des objets JSON horodatés.

5 DynamoDB — modèle d'accès et index secondaire global

5.1 Modèle de données

Chaque item DynamoDB [3] dans les trois tables temps réel porte un attribut `type` (valeur constante par table : "parkings", "traffic", "departures") et un `ingestion_timestamp` en microsecondes Unix. Ce couple constitue la clé du GSI. L'horodatage est exprimé en microsecondes (`timestamp() * 1_000_000`) pour garantir l'unicité sur des ingestions rapprochées : un horodatage en secondes provoquerait des collisions si deux snapshots arrivaient dans la même

seconde.

```

1 def create_stations_departures_dict(json_data, ingestion_timestamp_str):
2     results = []
3     for index, departure in enumerate(json_data["departures"]):
4         result = {
5             "gid": index,
6             "trip_id": departure["display_informations"]["trip_short_name"],
7             "ingestion_timestamp": int(
8                 datetime.fromisoformat(ingestion_timestamp_str).timestamp()
9                 * 1_000_000),          # microsecondes : unicite garantie
10            "departure_datetime": departure["stop_date_time"]["
11            base_departure_date_time"],
12            "departure_station": departure["stop_point"]["name"],
13            "network": departure["display_informations"]["network"]
14        ],
15        "type": "departures",          # PK du GSI : valeur constante par
16        "table":
17            "long": departure["stop_point"]["coord"]["lon"],
18            "lat": departure["stop_point"]["coord"]["lat"],
19        }
20        results.append(result)
21    return results

```

Extrait 2 – Création d'un item avec horodatage microseconde (process_s3_to_dynamodb.py)

5.2 Global Secondary Index — requête $O(1)$ sur le dernier snapshot

L'enjeu de performance est d'accéder au dernier snapshot *sans* scanner toute la table. Sans index, chaque requête « dernier état des parkings » serait un *scan* en $O(n)$ sur l'ensemble de l'historique. Avec le GSI `gsi-type-ingestion_timestamp` (PK : type, SK : `ingestion_timestamp`), une *query* avec `ScanIndexForward=False`, `Limit=1` retourne l'enregistrement le plus récent en $O(1)$.

Approche	Sans GSI (scan)	Avec GSI (query)
Items lus	Toute la table (N)	Exactement 1
Unités de lecture (RCU)	N	1
Complexité	$O(n)$	$O(1)$
Tri	Côté application	Natif (sort key)

```
def get_last_processed_timestamp(dynamodb, table_name, type_table):
    table = dynamodb.Table(table_name)
    try:
        response = table.query(
            IndexName='gsi-type-ingestion_timestamp',
            KeyConditionExpression=Key('type').eq(type_table),
            ScanIndexForward=False, # tri SK descendant : le plus grand en
            # tete
            Limit=1,
            ProjectionExpression="ingestion_timestamp"
        )
        items = response.get('Items', [])
        return items[0]['ingestion_timestamp'] if items else None
    except Exception:
        # Fallback : scan si GSI non disponible (degraded mode)
        response = table.scan(
            ProjectionExpression="ingestion_timestamp", Limit=1000)
        timestamps = [item['ingestion_timestamp'] for item in response['Items']]
    return max(timestamps) if timestamps else None
```

Extrait 3 – Requête GSI – dernier timestamp $O(1)$ avec fallback scan (utils_dynamodb.py)

Le fallback sur `scan` garantit la disponibilité du service même si le GSI est en cours de construction ou temporairement indisponible (mode dégradé). Ce même pattern — query GSI en priorité, scan en secours — est réutilisé dans le Lambda de l'API.

[Capture] Console AWS DynamoDB – table `parkings`, onglet `Indexes` : configuration du GSI `gsi-type-ingestion_timestamp`.

5.3 Ingestion incrémentale depuis S3

Le pipeline S3 → DynamoDB ne retraits que les nouveaux objets S3. La borne de reprise est le dernier `ingestion_timestamp` connu via le GSI ; les clés S3 sont filtrées par comparaison ISO de leur horodatage.

```
last_processed_timestamp_str = get_last_processed_timestamp(  
    dynamodb, DYNAMODB_TABLE_NAME, "departures")  
  
keys      = get_s3_object_keys(s3, BUCKET_NAME, s3_folder)  
new_keys  = []  
  
if last_processed_timestamp_str:  
    last_dt = datetime.fromtimestamp(  
        float(last_processed_timestamp_str) / 1_000_000)  
    for key in keys:  
        key_dt = datetime.fromisoformat(key.split('/')[1])  
        if key_dt > last_dt:  
            new_keys.append(key)  
else:  
    new_keys = keys      # premiere ingestion : tout traiter  
  
for key in new_keys:  
    response = get_json_from_s3(s3, BUCKET_NAME, key)  
    items    = create_stations_departures_dict(response, key.split('/')[1])  
    batch_put_items_to_dynamodb(dynamodb, DYNAMODB_TABLE_NAME, items)
```

Extrait 4 – Ingestion incrémentale S3 vers DynamoDB (process_s3_to_dynamodb.py)

L'idempotence est garantie : à chaque relance, seuls les objets S3 postérieurs au dernier horodatage connu sont réingérés, sans duplication des données déjà présentes.

6 Traitement cloud — agrégation analytique

Un pipeline d'agrégation lit les tables temps réel (Silver) et produit dix tables analytiques pré-agrégées dans DynamoDB (Gold). Pour les parkings, l'agrégation se fait par tranche horaire (groupement sur `ingestion_timestamp // 3_600_000_000`), puis calcul des indicateurs métier.

Table DynamoDB (Gold)	Indicateur
aggregation_overall_occupancy_rate	Taux d'occupation global (%)
aggregation_average_availability_parking	Disponibilité moyenne par parking
aggregation_reference_pricing	Tarification moyenne (1h/2h/4h/24h)
aggregation_number_of_parkings_in_operation	Nombre de parkings ouverts
aggregation_traffic_congestion_index	Indice de congestion du trafic
aggregation_traffic_critical_segments	Segments routiers critiques
aggregation_traffic_fluidity_by_zone	Fluidité par zone géographique
aggregation_departures_by_network	Départs par réseau (SNCF/RATP)
aggregation_departures_top_destinations	Top destinations
aggregation_departures_total	Total des départs sur la période

Ce pré-calcul transfère la charge CPU du moment de la requête vers le moment de l'ingestion : l'API lit ensuite ces tables d'agrégats (petites et statiques) sans recalculer quoi que ce soit à chaque appel.

7 Mise à disposition — Lambda et API Gateway

7.1 Routage des 13 endpoints serverless

L'API expose treize endpoints via un Lambda [4] unique routé par chemin : trois endpoints temps réel (requête GSI vers le dernier snapshot) et dix endpoints d'agrégats (scan de tables statiques).


```
def lambda_handler(event, context):
    dynamodb = boto3.resource('dynamodb')
    path = event.get('path')

    # Endpoints agregats : scan complet (tables fixes, volume faible)
    aggregation_tables = [
        "aggregation_overall_occupancy_rate",
        "aggregation_average_availability_parking",
        "aggregation_reference_pricing",
        "aggregation_traffic_congestion_index",
        "aggregation_departures_by_network",
        # ... 10 tables au total
    ]

    if path.lstrip('/') in aggregation_tables:
        items = get_all_items(dynamodb, path.lstrip('/'))
        return {'statusCode': 200,
                'body': json.dumps(items, default=decimal_serializer)}

    # Endpoints temps reel : requete GSI -> dernier snapshot
    if path == '/parkings': table_name, type_value = "parkings", "parkings"
    elif path == '/traffic': table_name, type_value = "traffic", "traffic"
    elif path == '/departures': table_name, type_value = "departures", "departures"
    else:
        return {'statusCode': 404,
                'body': json.dumps({'message': f'Not Found {path}'})}

    last_ts = get_last_ingestion_time(dynamodb, table_name, type_value)
    items = get_items_by_ingestion_time(dynamodb, table_name, last_ts, type_value)
    return {'statusCode': 200,
            'body': json.dumps(items, default=decimal_serializer)}
```

Extrait 5 – Lambda handler – routage des 13 endpoints (api/main.py)

L'architecture serverless (Lambda + API Gateway [5]) supprime toute gestion d'infrastructure : pas de serveur à provisionner, pas de scaling manuel, et une facturation à la requête — optimale pour un flux temps réel à appels intermittents.

7.2 Cold start et stratégies de mitigation

Une invocation Lambda comprend deux phases : l'initialisation (*cold start*) et l'exécution. Le cold start n'a lieu qu'à la première invocation après une période d'inactivité ; pour un Lambda Python avec boto3, il est typiquement de 200 à 500 ms.

Phase	Durée typique	Fréquence
Cold start (init)	200–500 ms	Première invocation, après inactivité
Warm execution	10–100 ms	Invocations suivantes (même conteneur)

Pour un tableau de bord rafraîchi toutes les 5 minutes, le cold start est acceptable. Pour une API à très faible latence, trois leviers existent : la **Provisioned Concurrency** (instances maintenues chaudes en permanence), un **warm-up ping** EventBridge, ou un package minimal (boto3 étant fourni par le runtime AWS, le ZIP ne contient que le code applicatif ce qui réduit le temps de chargement).

[Capture] Console AWS API Gateway – liste des 13 routes configurées (méthode GET + intégration Lambda).

8 Sécurité et gestion des identités

8.1 Principe du moindre privilège (IAM)

Chaque composant s'exécute sous un rôle IAM dédié n'accordant que les permissions strictement nécessaires à sa fonction.

Rôle IAM	Permissions	Justification
lambda-api-role	dynamodb:GetItem dynamodb:Query dynamodb:Scan	L'API ne fait que lire — aucune permission d'écriture accordée.
ingestion-role	s3:PutObject dynamodb:PutItem dynamodb:BatchWriteItem	L'ingestion écrit S3 et DynamoDB, sans relecture.
aggregation-role	dynamodb:Query (Silver) dynamodb:PutItem (Gold)	Isolation stricte : lecture Silver, écriture Gold.

Cette séparation garantit qu'une compromission d'un Lambda d'agrégation ne peut ni écrire dans les tables Silver ni lire d'autres tables Gold que les siennes. En production, les rôles seraient affinés par préfixe de table via des conditions IAM `StringLike` sur l'ARN.

8.2 Chiffrement des données

Le chiffrement en transit est assuré par défaut en HTTPS (TLS 1.2+) sur tous les appels boto3. Le chiffrement au repos est natif sur chaque service managé.

Service	Mode de chiffrement	Implémentation projet
AWS S3	SSE-S3 (clé gérée AWS)	Chiffrement côté serveur automatique
AWS DynamoDB	AES-256 (clé AWS)	Activé par défaut sur toutes les tables
API Gateway	TLS 1.2+ obligatoire	Aucun trafic HTTP non chiffré

Pour des données sensibles (positions GPS, données personnelles de transport), le SSE-KMS avec rotation annuelle de clé serait retenu : il permet de révoquer l'accès en révoquant la clé, sans supprimer les objets.

8.3 Automatisation de l'ingestion par EventBridge

L'ingestion manuelle actuelle serait industrialisée par AWS EventBridge Scheduler : un schedule cron déclenche chaque Lambda d'ingestion toutes les 5 minutes.

```
aws scheduler create-schedule \
  --name "ingest-parkings-every-5min" \
  --schedule-expression "rate(5 minutes)" \
  --target '{
    "Arn": "arn:aws:lambda:eu-west-1:ACCOUNT:function:ingest-parkings",
    "RoleArn": "arn:aws:iam::ACCOUNT:role/scheduler-invoke-role"
  }' \
  --flexible-time-window '{"Mode": "OFF"}'
```

Extrait 6 – EventBridge Scheduler – déclenchement périodique (AWS CLI)

Le rôle `scheduler-invoke-role` ne dispose que de la permission `lambda:InvokeFunction` sur la fonction ciblée — nouvelle application du moindre privilège.

9 Optimisation des performances et des coûts

9.1 Serverless contre serveur permanent

L'architecture Lambda + API Gateway est coût-efficace pour un flux intermittent. La comparaison avec une instance EC2 permanente illustre le gain.

Critère	Lambda + API Gateway	EC2 t3.small
Facturation	À la requête (0,20 \$/M appels)	À l'heure (≈18 \$/mois)
Scaling	Automatique, instantané	Manuel ou auto-scaling (délai)
Coût en veille	0 \$	18 \$/mois (actif 24/7)
Maintenance OS	Aucune (managé)	Patches, mises à jour, supervision

Pour le profil du projet (appels toutes les 5 minutes), la facturation Lambda est négligeable : ≈ 8 640 invocations/mois, très en dessous du palier gratuit AWS (1 million d'invocations/mois).

9.2 Cycle de vie du stockage S3

Le bucket accumule des snapshots toutes les 5 minutes ; sans politique de rétention, le coût de stockage croît indéfiniment. Une politique de cycle de vie S3 automatise l'archivage puis la suppression.

Règle	Délai	Effet
Transition vers S3 Infrequent Access	30 jours	−40 % de coût
Transition vers S3 Glacier	90 jours	−70 % (récupération lente)
Suppression automatique	365 jours	Coût nul

9.3 Supervision CloudWatch

CloudWatch collecte automatiquement les métriques de chaque service. Une alarme sur `ThrottledRequests` > 0 déclenche une notification SNS, permettant de réagir avant que le throttling n’affecte les utilisateurs.

Service	Métrique	Seuil d’alerte
Lambda ingestion	Errors	> 0 erreur en 5 min
Lambda API	Duration	> 3 secondes
DynamoDB	ThrottledRequests	> 0 (critique)
API Gateway	5XXError	> 1 % du trafic
S3	NumberOfObjects	Croissance anormale

[Capture] Tableau de bord Streamlit – page Parkings : taux d’occupation global en temps réel et carte des parkings disponibles.

10 Synthèse — compétences démontrées

Le projet cloud couvre l’intégralité du périmètre du Bloc 3 à travers une architecture serverless AWS complète, du data lake brutes à l’API de consommation.

10.1 Preuves d’acquisition des compétences

Le tableau suivant récapitule, pour chacune des quatre sous-compétences du Bloc 3, la réalisation concrète qui la démontre dans le projet.

Compétence visée	Réalisation et preuve dans le projet
Stockage de données dans le cloud	Lac de données S3 horodaté (Bronze) et base DynamoDB managée (Silver/Gold), avec choix de services justifié face aux alternatives AWS.
Pipelines de traitement cloud	Ingestion incrémentale idempotente S3 → DynamoDB et pipeline d'agrégation produisant 10 tables analytiques pré-calculées.
Sécurité (chiffrement & IAM)	Rôles IAM au moindre privilège, chiffrement au repos (SSE-S3, AES-256) et en transit (TLS 1.2+), automatisation sécurisée via EventBridge.
Optimisation (performance, disponibilité, coûts)	GSI pour un accès $O(1)$, compute serverless facturé à l'usage, cycle de vie S3 et supervision CloudWatch (métriques + alarmes SNS).

Références

- [1] France Compétences. *Fiche RNCP36739 — Expert en ingénierie de données*. Disponible sur : <https://www.francecompetences.fr/recherche/rncp/36739/> [consulté le 15 juin 2026].
- [2] Amazon Web Services. *Amazon S3 — Object Storage*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/s3/> [consulté le 15 juin 2026].
- [3] DeCandia G., Hastorun D., Jampani M. *et al.* *Dynamo : Amazon's Highly Available Key-value Store*. *ACM SIGOPS Operating Systems Review*, vol. 41, n°6, pp. 205–220, 2007.
- [4] Amazon Web Services. *AWS Lambda — Serverless Compute*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/lambda/> [consulté le 15 juin 2026].
- [5] Amazon Web Services. *Amazon API Gateway*. Documentation officielle AWS. Disponible sur : <https://docs.aws.amazon.com/apigateway/> [consulté le 15 juin 2026].